

Technische Hochschule Ingolstadt
Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

Message Passing Networks

CNNs for Tracking

Dec 04, 2025

Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler

Technische Hochschule Ingolstadt

AIMotion, AI and Sensor Data Fusion

Email: bjoern.giesler@thi.de

Sensor Networks and Sensor Fusion

Organizational Matters

- **Please register for the lecture on Moodle!**
  - Link: https://moodle.thi.de/course/view.php?id=10580
  - Key: **SensNT+DF#WS25**
- **News on Moodle:**
  - Slides for the lecture (upcoming lectures marked as WIP and are subject to change!)
  - Formulary, Exercises (Solutions provided after discussion in the lecture)
  - Exercises are optional, one task / sheet per week, but are highly recommended.
    They can be handed in for correction.
  - I will *try* to publish exercises before Wednesday‘s lecture so that initial questions can be discussed on Thursday.
- **Exam: Written exam, 90 minutes**
- **Consultation hours: Please arrange via e-mail to bjoern.giesler@thi.de.**
- **Please also don‘t hesitate to send questions, lecture feedback, change requests, etc.!**

- **In the last lectures we looked at various forms of the Kalman filter – linear, extended, unscented, and explored what to do if we can't *model* a state transition function.**
- **Now we are in the middle of multi-object tracking applications.**
- **Last lecture: Network Flow Optimization and other global techniques**
- **This lecture: DNNs for tracking**

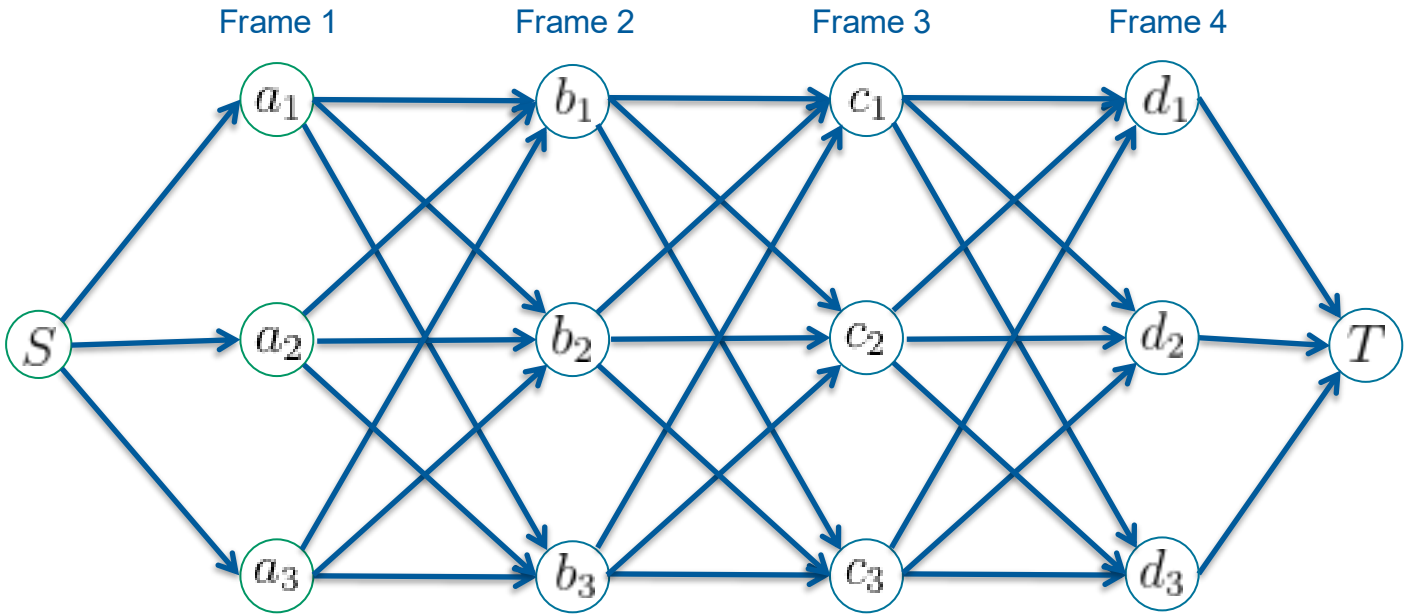
- **Going forward: Particle Filters and similar inherent multi hypothesis approaches.**
- **After that, we will revisit tracking briefly using Joint Probability Data Association Filters.**
- **The rest of the lecture series will deal with**
  - Simultaneous Localization and Mapping, and
  - (DNN-based) early fusion approaches.

- **Like every semester, we are conducting a round of lecture / teaching feedback.**
- **Link is in Moodle, but you can also use the QR code to the right.**
- **Link gets you to the German version, but you can choose the English version (link @top right)**
- **I highly encourage feedback**
  - direct if possible (because it allows asking for clarification)
  - but this is cool too.
- **Please do participate, I'm happy to read your opinion.**



Tracking as a Graph Optimization Problem (cont'd)

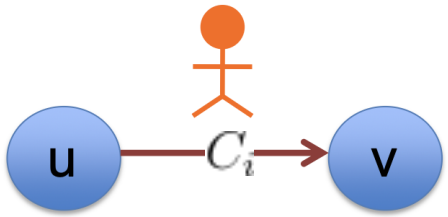
Using Network Flow For Tracking



- **Approach:**
  - Seek a globally optimal solution by considering observations over all frames in "batch mode".
  - Extend 2-frame min-cost formulation by adding observations from all frames into the network.

Using Network Flow For Tracking [1]

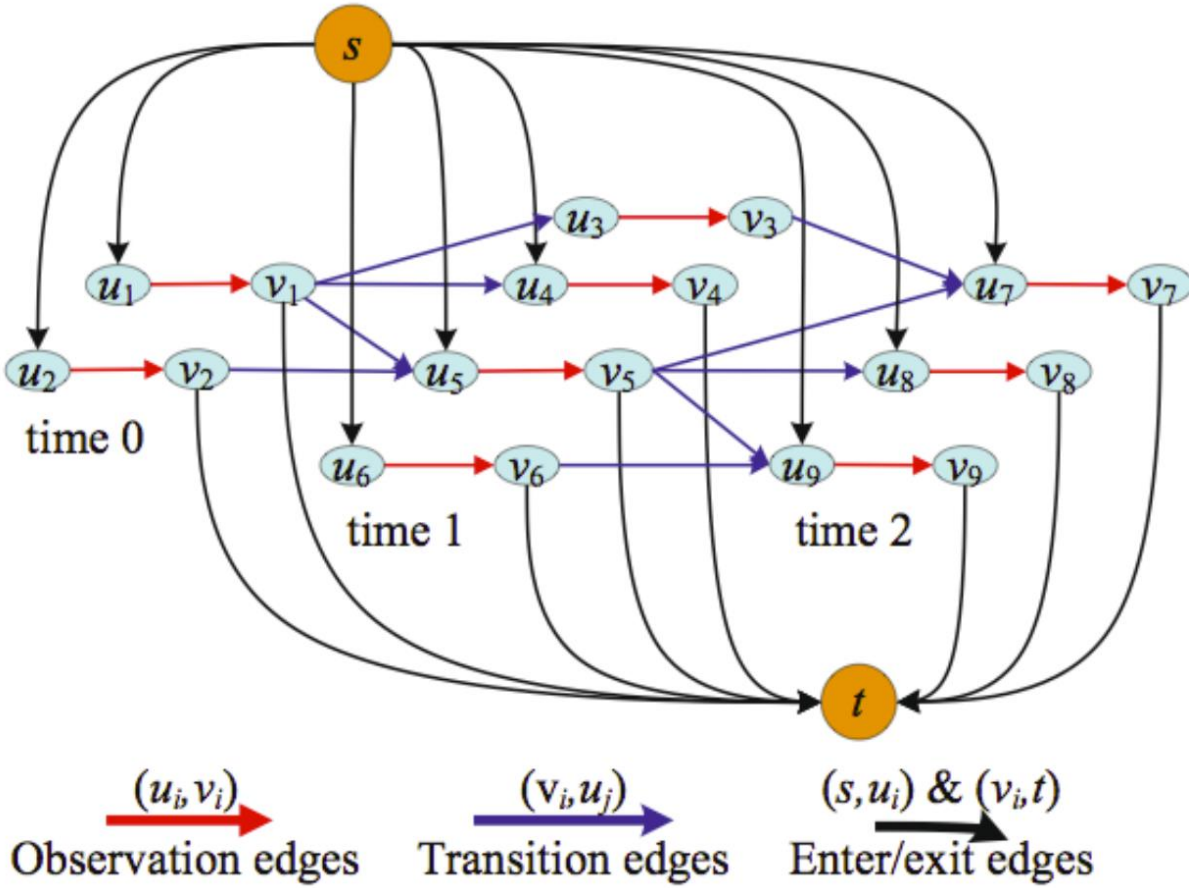
- **Divide everv detection into 2 nodes**



Detection edge

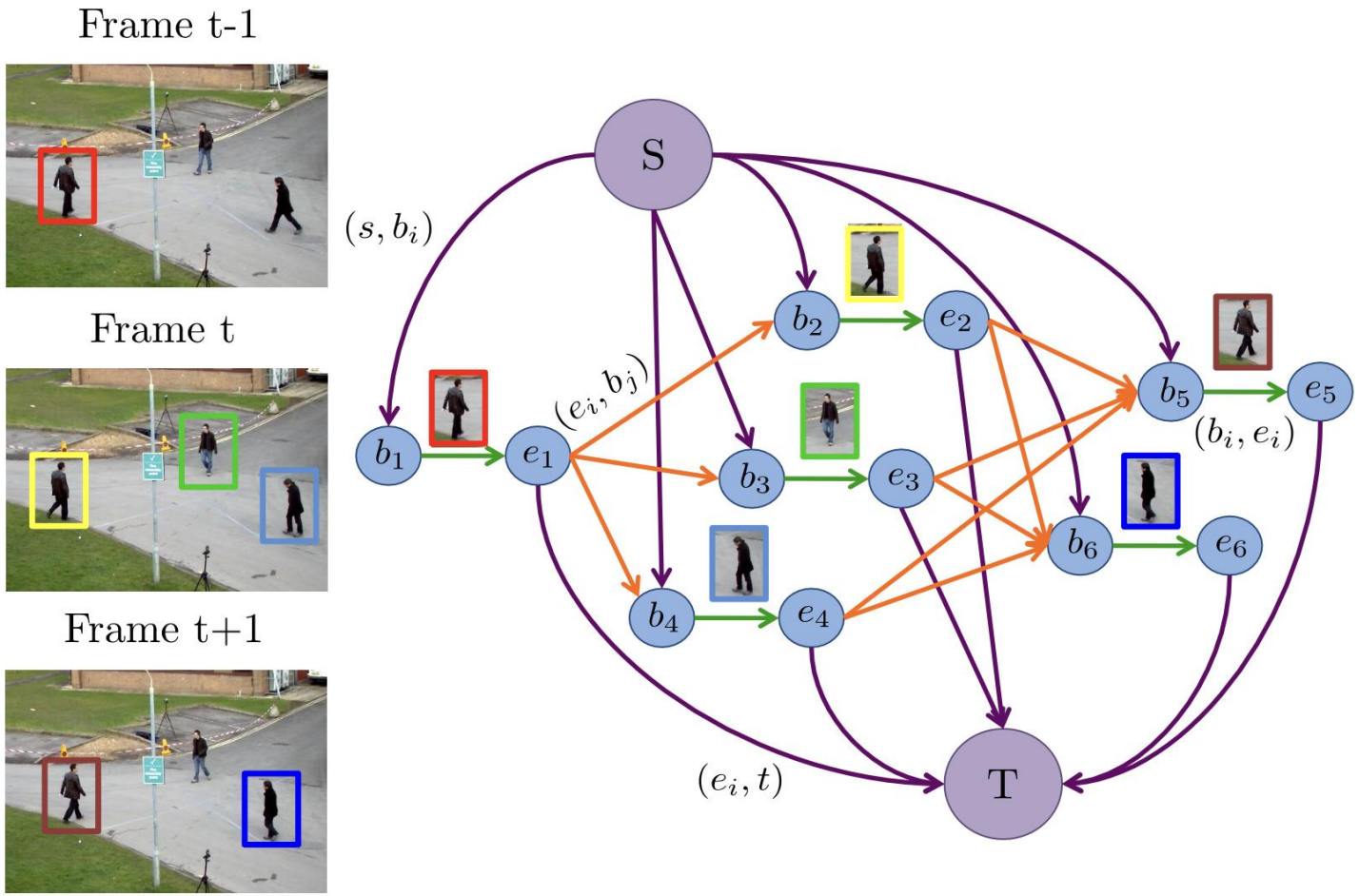
$$C_i = \log \frac{\beta_i}{1-\beta_i} = -\log \frac{1-\beta_i}{\beta_i} = -\log(P_i)$$

with β_i the likelihood that detection i is a false alarm



from [1]

Using Network Flow For Tracking [1]



Minimum Cost Formulation

- **Minimize**

$$\mathcal{T}^* = \underset{\mathcal{T}}{\operatorname{argmin}} \sum_i C_{\text{in},i} f_{\text{in},i} + \sum_i C_{i,\text{out}} f_{i,\text{out}} + \sum_{i,j} C_{i,j} f_{i,j} + \sum_i C_i f_i$$

subject to

- flow conservation at all nodes: $\forall i: \; f_{\text{in},i} + \sum_j f_{j,i} = f_i = f_{\text{out},i} + \sum_j f_{i,j}$

- Edge capacities: $f_i \leq 1$

Minimum Cost Formulation

- **Objective Function**

$$\mathcal{T}^* = \underset{\mathcal{T}}{\operatorname{argmin}} \sum_i C_{\text{in},i} f_{\text{in},i} + \sum_i C_{i,\text{out}} f_{i,\text{out}} + \sum_{i,j} C_{i,j} f_{i,j} + \sum_i C_i f_i$$

$$\downarrow \quad C_i = -\log(P_i)$$

is equivalent to the Maximum A-Posteriori formulation

$$\mathcal{T}^* = \underset{\mathcal{T}}{\operatorname{argmax}} \prod_i P(\mathbf{o}_i|\mathcal{T}) P(\mathcal{T})$$

Minimum Cost Formulation

- **Objective Function**

$$\mathcal{T}^* = \underset{\mathcal{T}}{\operatorname{argmin}} \underbrace{\sum_i C_{\text{in},i} f_{\text{in},i}}_{\text{In}} + \underbrace{\sum_i C_{i,\text{out}} f_{i,\text{out}}}_{\text{Out}} + \underbrace{\sum_{i,j} C_{i,j} f_{i,j}}_{\text{Transition}} + \underbrace{\sum_i C_i f_i}_{\text{Detection Likelihood}}$$

is equivalent to the Maximum A-Posteriori formulation

$$\mathcal{T}^* = \underset{\mathcal{T}}{\operatorname{argmax}} \prod_i P(\mathbf{o}_i|\mathcal{T}) P(\mathcal{T})$$

$$P(\mathcal{T}) = \prod_{T_t \in \mathcal{T}} P(T_t)$$

Independence Assumption
+ Markov Assumption

Tracking as Network Flow Optimization: Summary

- **Pros**
  - Clear algorithmic framework
  - Equivalence to probabilistic formulation
  - Well-understood Linear Programming optimization problem, efficient algorithms available
  - Globally optimal solution
- **Cons / Limitations**
  - Only applicable to restricted problem setting due to Linear Programming formulation
    - Not possible to encode exclusion constraints between detections (e.g. to penalize physical overlap)
    - Motion model can only draw upon information from pairs of detections (i.e. no constant velocity models)
  - C_{in} and C_{out} fiddly to set in practice (too low ➔ fragmentations, too high ➔ ID switches)

Message Passing Networks

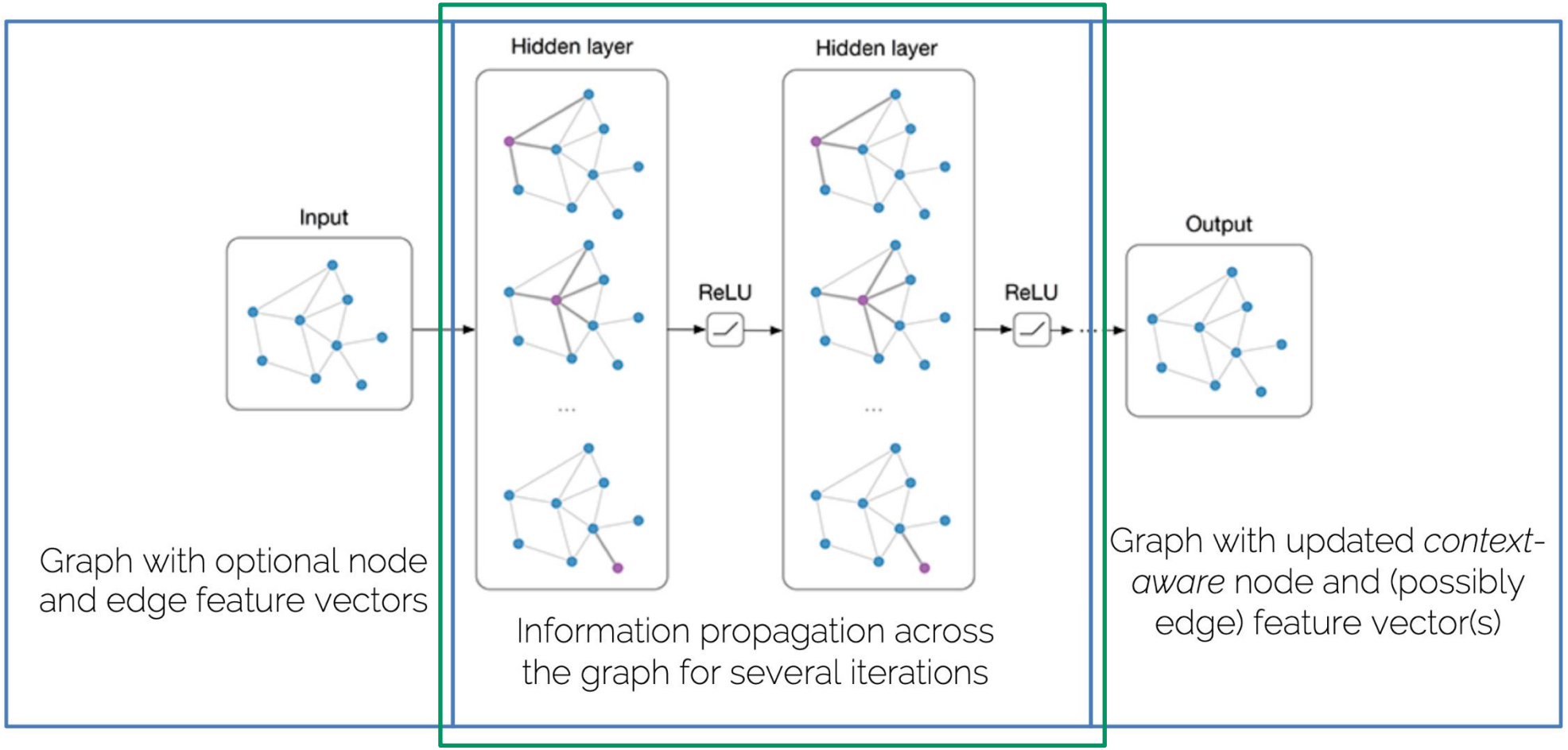
Graph CNNs

Slides credit: Robert Collins / Bastian Leibe

Graph CNNs

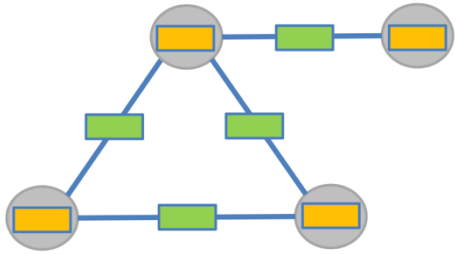
- **Important real-world datasets take the form of graphs: Social networks, knowledge graphs, WWW, etc.**
- **Several works have proposed generalizations of Neural Networks that can operate on graph-structured domains:**
  - Scarselli et al. “The Graph Neural Network Model”. IEEE Trans. Neur. Net 2009.
  - Kipf et al. “Semi-Supervised Classification with Graph Convolutional Networks. ICLR 2016.
  - Gilmer et al. “Neural Message Passing for Quantum Chemistry”. ICML 2017
  - Battaglia et al. “Relational inductive biases, deep learning, and graph networks”. arXiv 2018 (review paper)
- **Key challenges:**
  - Variable sized inputs (number of nodes and edges)
  - Need invariance to node permutations

Graph CNNs: General Idea

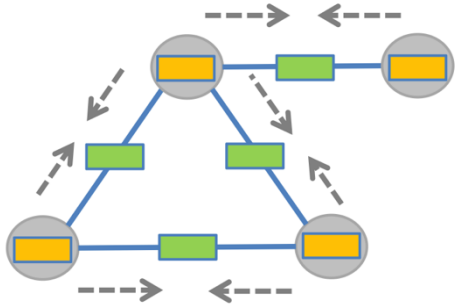


Graph CNNs: Propagating Information

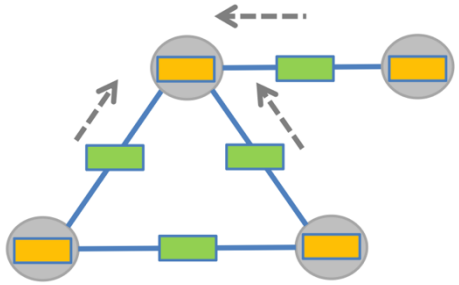
- **We can divide the propagation process into two steps:**
  - "Node to Edge" updates
  - "Edge to Node" updates



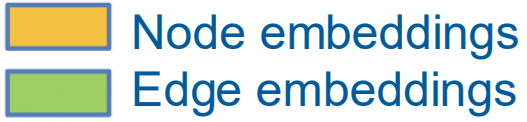
Initial Graph



Node to Edge Update



Edge to Node Update



Repeat these two updates for a fixed number of iterations
("message passing steps")
in order to encode context into the embeddings.

"Node to Edge" Updates

- **Notation:**
  - Graph: $G = (V, E)$
  - Initial embeddings: $h^{(0)}_{(i,j)},\ (i,j) \in E$ Embedding of edge that connects nodes i and j

    $h^{(0)}_i,\ i \in V$ Embedding of node i
  - Embeddings after l steps: $h^{(l)}_{(i,j)},\ (i,j) \in E$ Edge and Node networks $\mathcal{N}_e, \mathcal{N}_v$

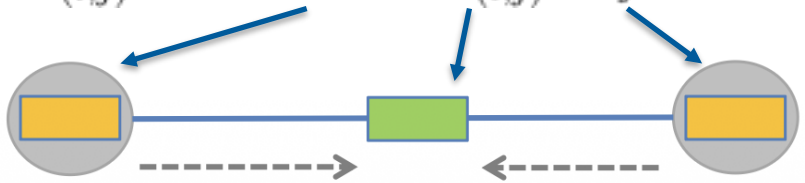
    $h^{(l)}_i,\ i \in V$

"Node to Edge" Updates

- **Notation:**
  - Graph: $G = (V, E)$
  - Initial embeddings: $h^{(0)}_{(i,j)},\ (i,j) \in E$ Embedding of edge that connects nodes i and j

    $h^{(0)}_i,\ i \in V$ Embedding of node i
  - Embeddings after l steps: $h^{(l)}_{(i,j)},\ (i,j) \in E$ Edge and Node networks $\mathcal{N}_e, \mathcal{N}_v$

    $h^{(l)}_i,\ i \in V$
  - At every message passing step, first do: $h^{(l)}_{(i,j)} = \mathcal{N}_e([h^{(l-1)}_i, h^{(l-1)}_{(i,j)}, h^{(l-1)}_j])$

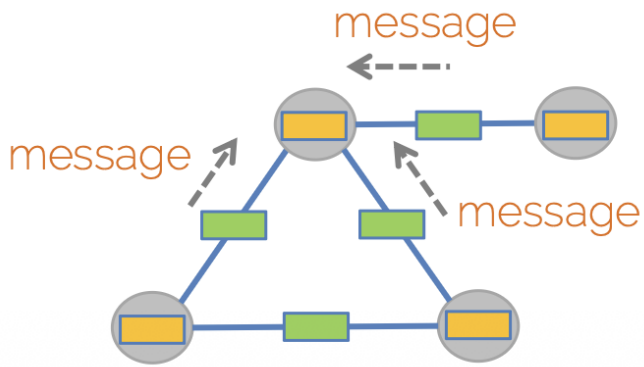


Combine node and edge embeddings

"Edge to Node" updates

- **After a round of edge updates, each edge embedding contains information about its pair of incident nodes.**
- **Then, edge embeddings are used to update nodes:**

$$m_{(i,j)}^{(l)} = \mathcal{N}_v([h_i^{(l-1)}, h_{(i,j)}^{(l)}])$$



"Edge to Node" updates

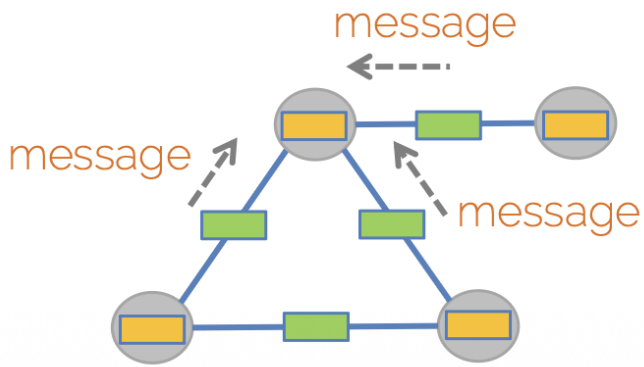
- **After a round of edge updates, each edge embedding contains information about its pair of incident nodes.**
- **Then, edge embeddings are used to update nodes:**

$$m_{(i,j)}^{(l)} = \mathcal{N}_v([h_i^{(l-1)}, h_{(i,j)}^{(l)}])$$

$$h_i^{(l)} = \Phi\left(\{m_{(i,j)}^{(l)}\}_{j \in N_i}\right)$$

Permutation-invariant operation (e.g. sum, mean, max)

Neighbors of node i



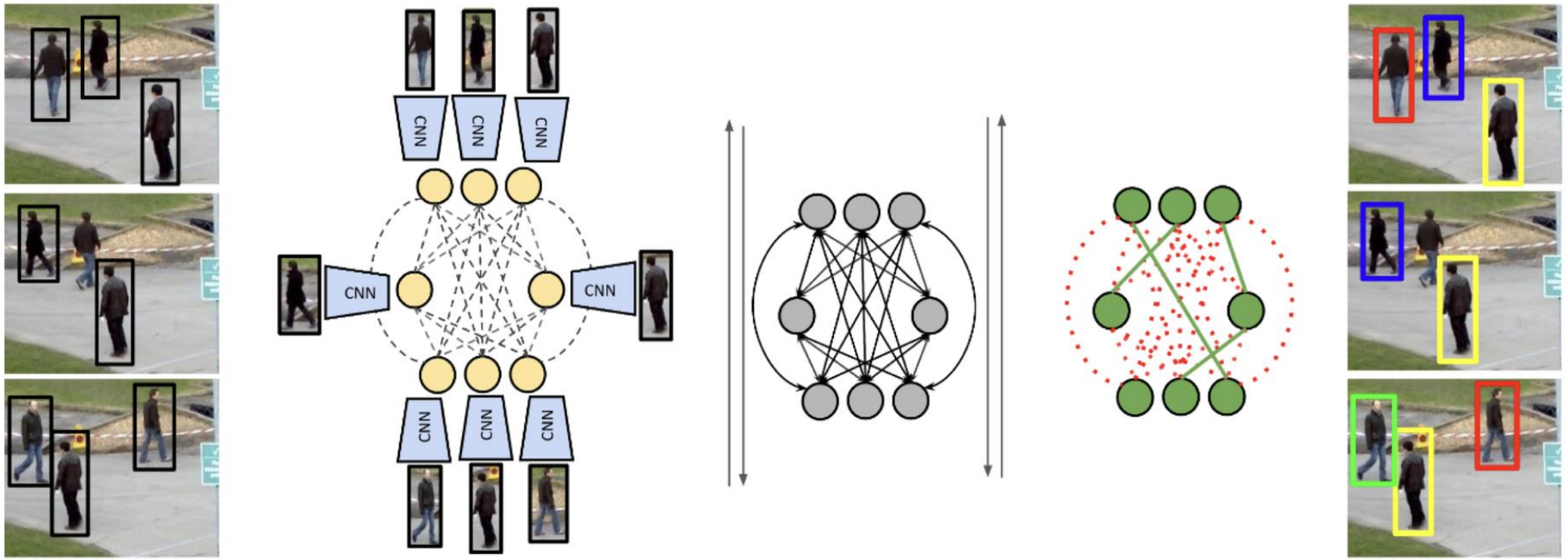
Message Passing Networks - Remarks

- **Main goal: obtain node and edge embeddings that contain *context information* encoding**
  - graph topology and
  - neighbor feature information.
- **After repeating the node and edge updates for l steps,**
  each node / edge embedding contains information about all nodes / edges at distance l.
- **You can think of iterations as layers in a CNN.**
- **Observe that all operations used are differentiable ➔ Message Passing Networks can be used within end-to-end learning pipelines.**
- **There is vast literature on different instantiations, as well as variations of the MPN framework we presented.**
- **See [1] for an extensive review.**

Multi-Object Tracking with Message Passing Networks

Slides credit: Laura Leal-Taixé, TUM

Overview

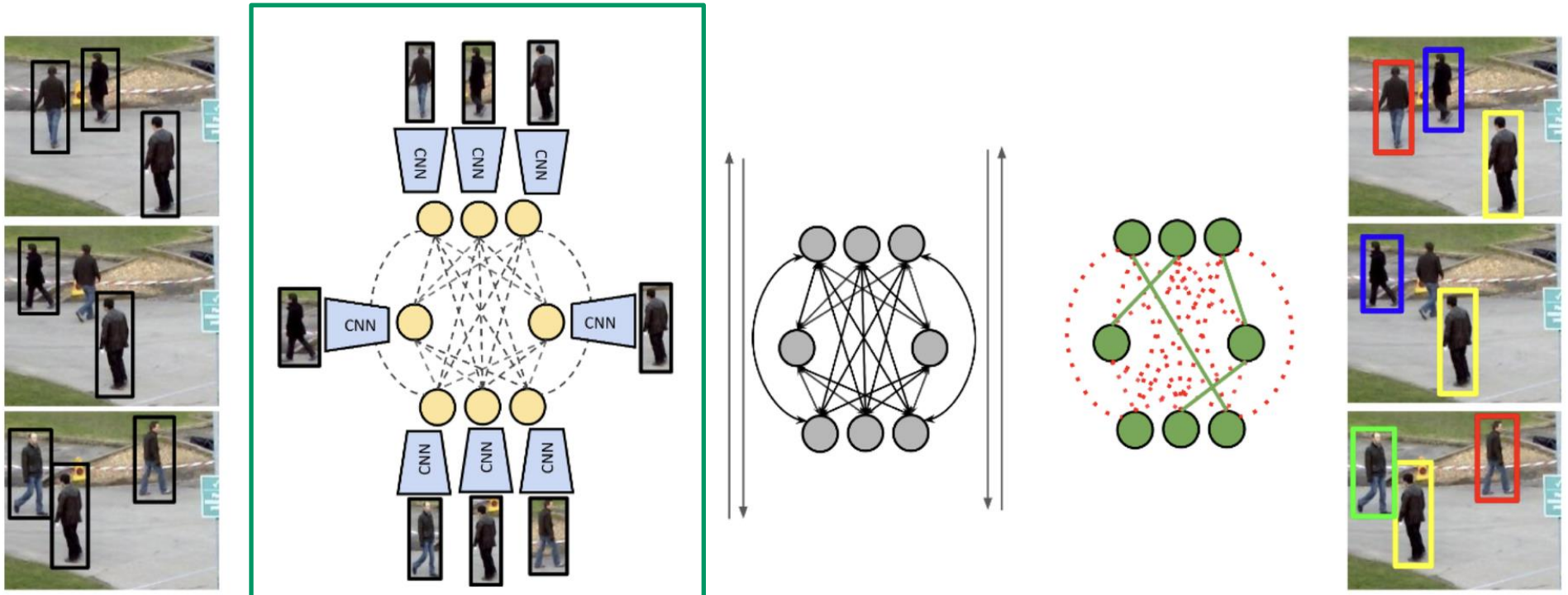


(a) Input (b) Graph Construction + Feature Encoding (c) Neural Message Passing (d) Edge Classification (e) Output

From [2]

Overview

Use a CNN to encode
appearance and scene geometry cues
into node and edge embeddings.

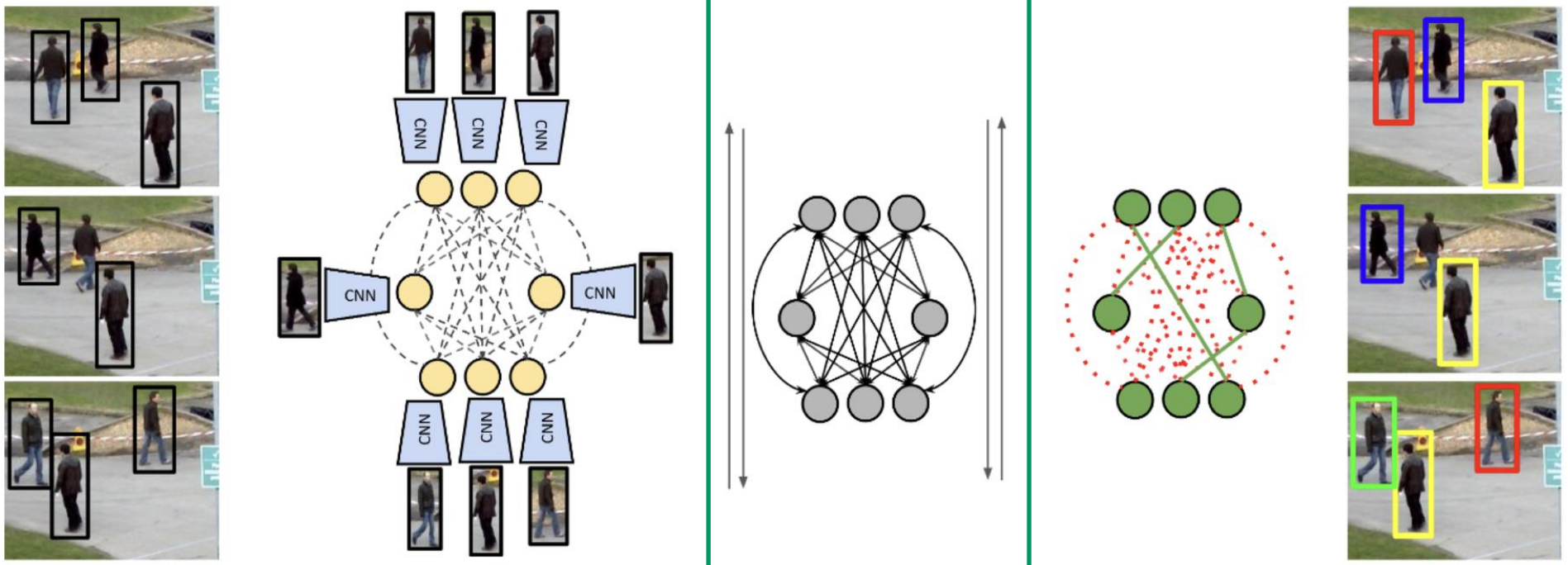


(a) Input (b) Graph Construction + Feature Encoding (c) Neural Message Passing (d) Edge Classification (e) Output

From [2]

Overview

Propagate cues across the entire graph with neural message passing.

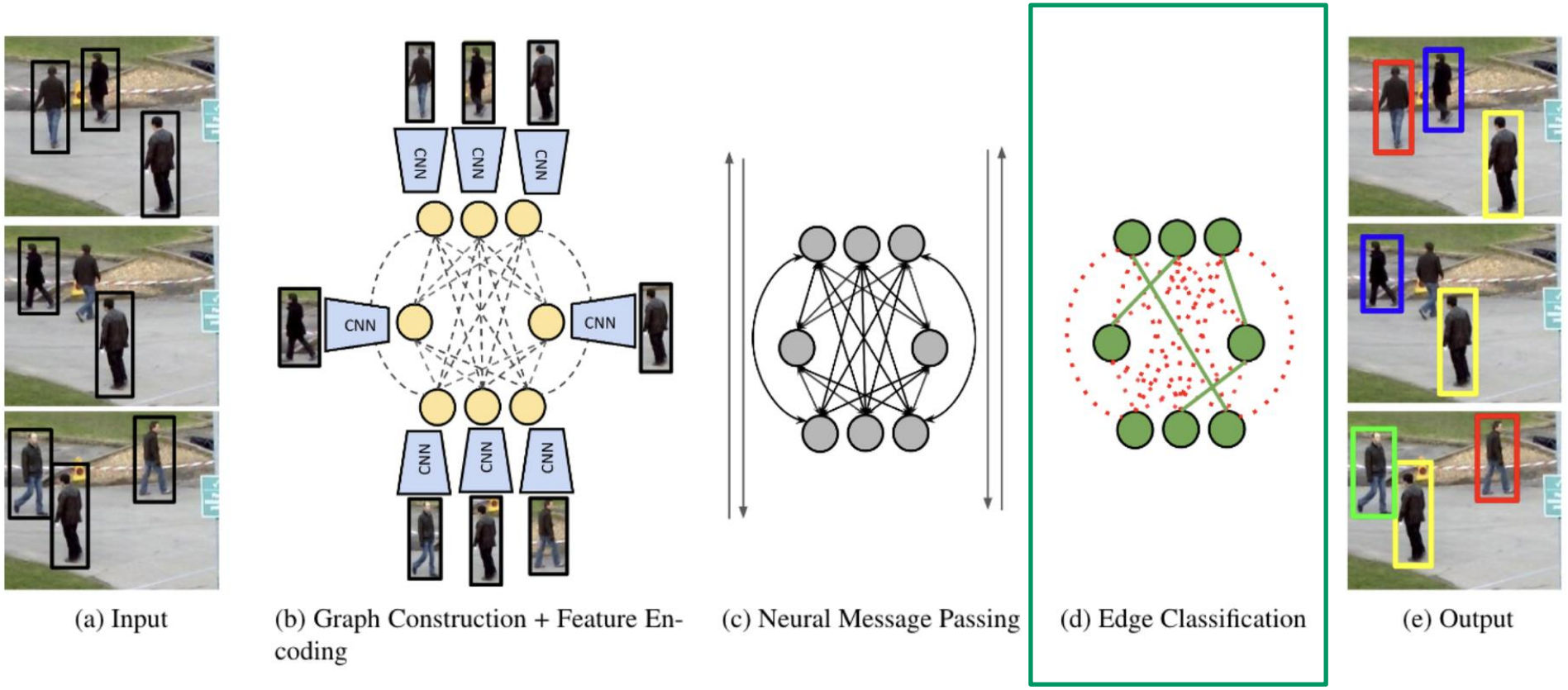


(a) Input (b) Graph Construction + Feature Encoding (c) Neural Message Passing (d) Edge Classification (e) Output

From [2]

Overview

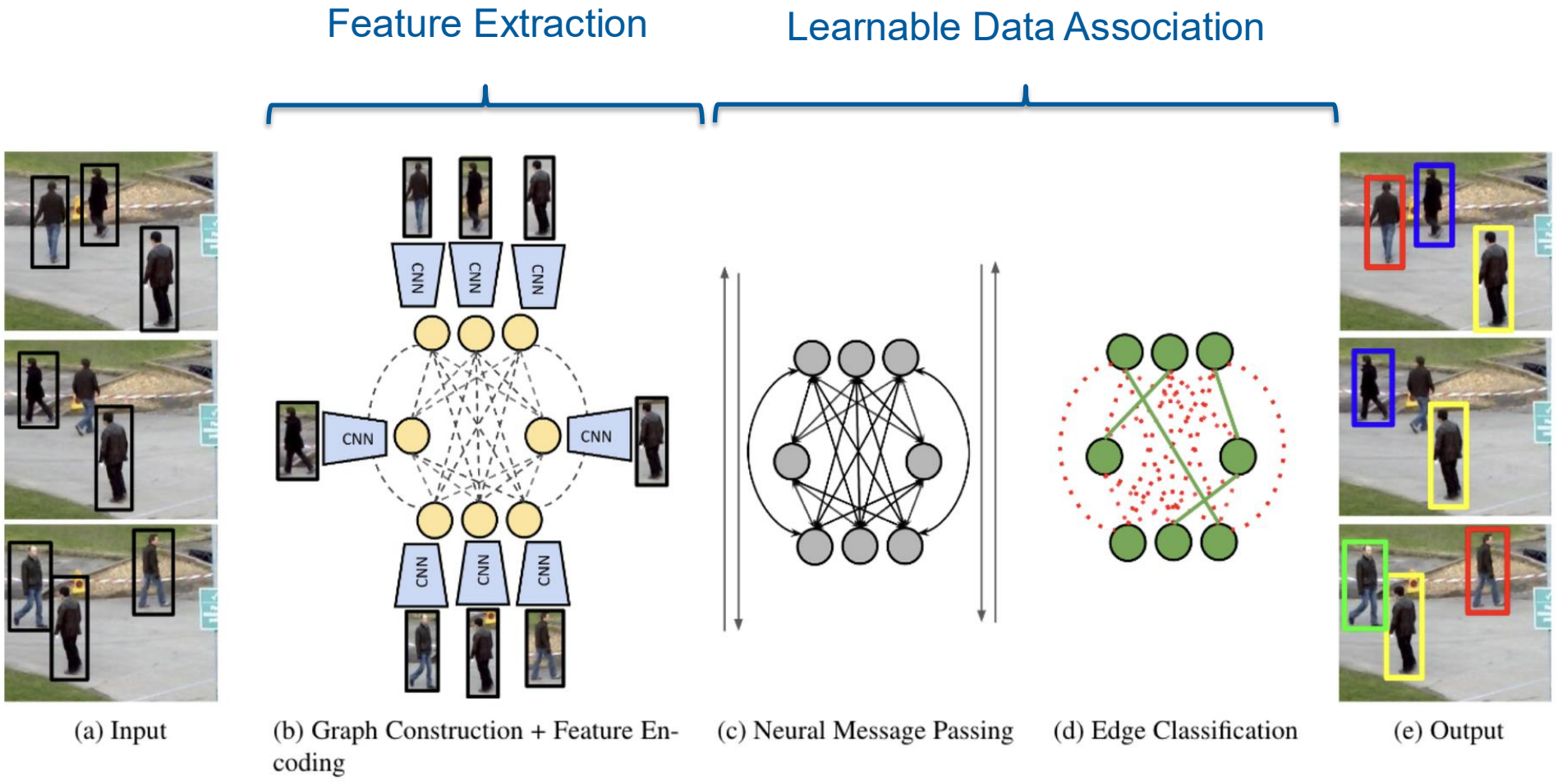
Learn to directly predict solutions of the min-cost flow problem by classifying edge embeddings.



(a) Input (b) Graph Construction + Feature Encoding (c) Neural Message Passing (d) Edge Classification (e) Output

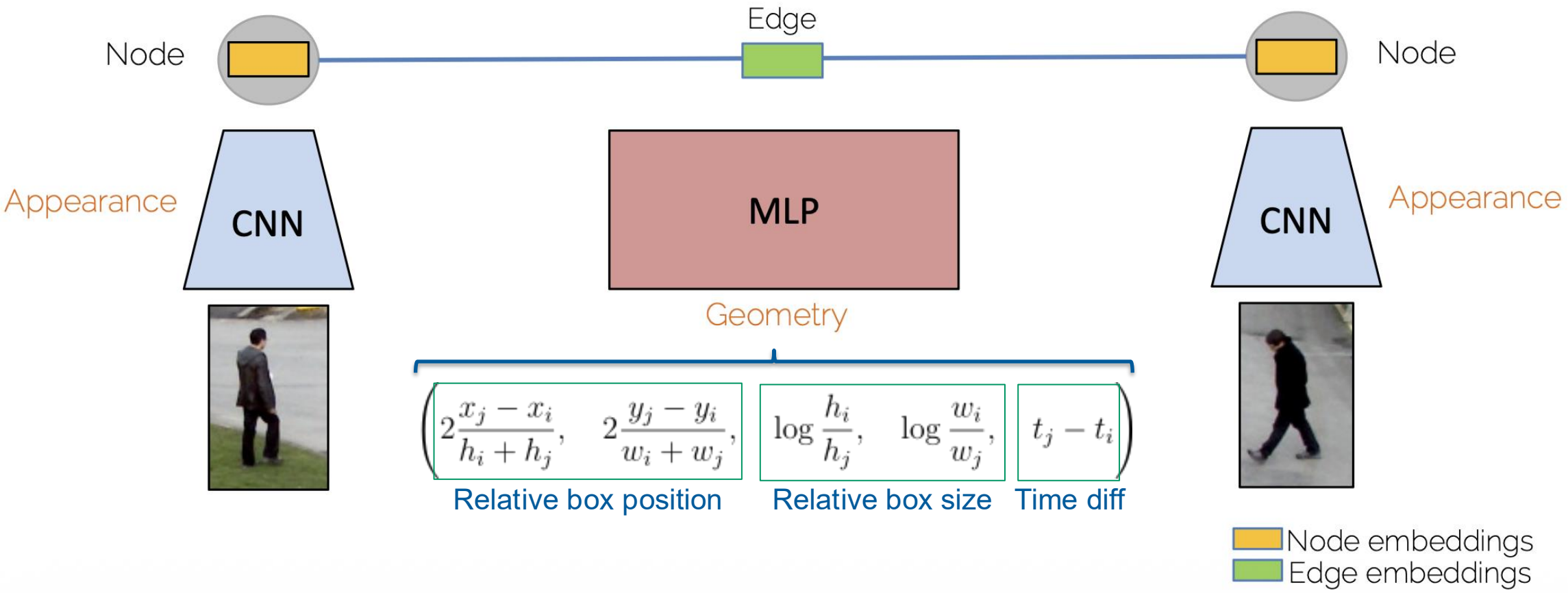
From [2]

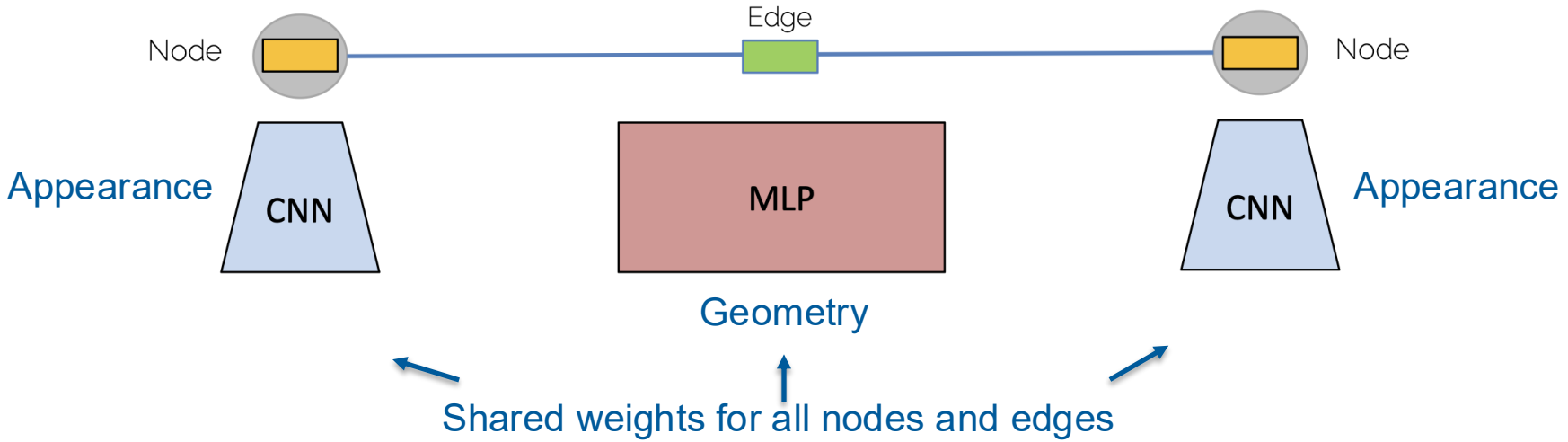
Overview



From [2]

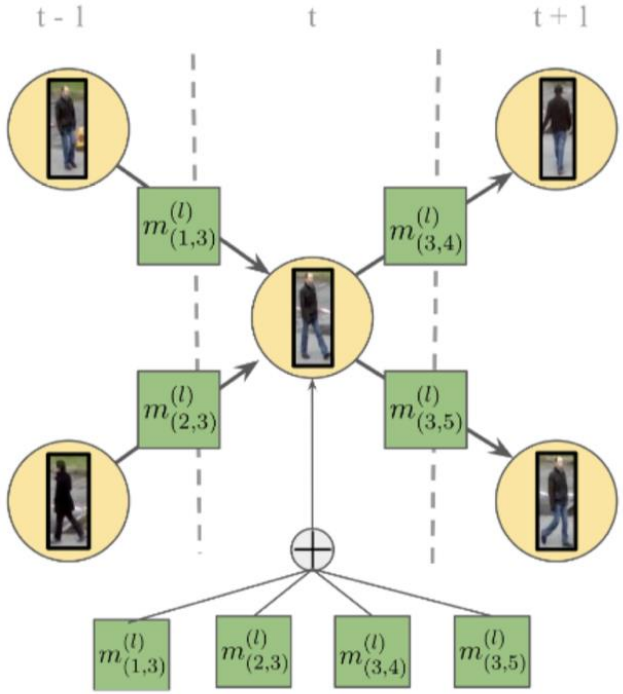
Feature Encoding





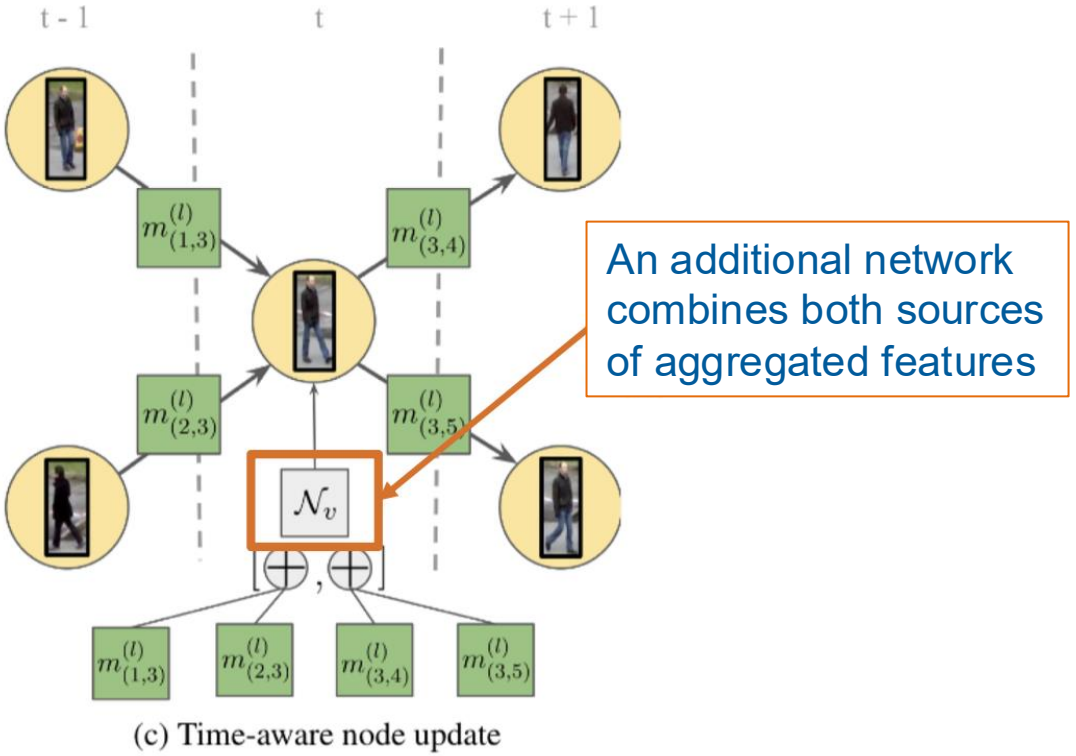
- **Instead of defining pairwise costs for edges and unary costs for nodes (classical setting), feature vectures encoding appearance and geometry cues are used.**
- **Goal: Propagate these embeddings across the entire graph in order to obtain new embeddings encoding high-order information among detections.**

Time-Aware Message Passing



(b) Vanilla node update

All node embeddings are aggregated at once

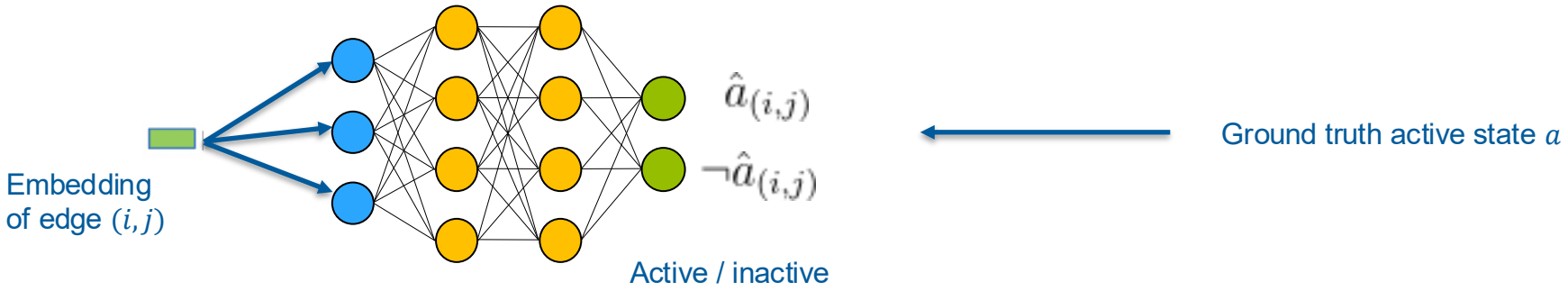


(c) Time-aware node update

Aggregation of nodes is separated between past / future frames

Classifying Edges

- **After several iterations of message passing, each edge embedding contains high-order information about other detections.**
- **We feed the embeddings to an MLP that predicts whether an edge is active/inactive.**



- **The loss function used for training is**

$$\mathcal{L} = -\frac{1}{|E|}\sum_{l=l_0}^{L}\sum_{(i,j)\in E} w \cdot a_{(i,j)} \log \hat{a}_{(i,j)}^{(l)} + (1 - a_{(i,j)} \log\left(1 - \hat{a}_{(i,j)}^{(l)}\right)$$

Sum over the last steps

Weight to balance active / inactive edges

Binary cross-entropy

Obtaining final results

- After classifying edges, we get a prediction between 0 and 1 for each edge in the graph.
- Directly thresholding solutions could yield infeasible solutions, although this is not frequent in practice.
- A simple rounding scheme (greedy or LP based) can be used to obtain final trajectories.
- In practice, around 99% of constraints are automatically satisfied, and rounding takes negligible time.
- The overall method is fast (~12 fps) and achieves State-of-the-Art in the MOT Challenge by a significant margin.

Evaluation of Multi-Object Tracking Solutions

Evaluation Metrics

- **Compute a set of measures per frame, performing matching between predictions and ground truth.**
- **Essential evaluation metrics:**
  - False Positive tracks (FPs) – seeing things that don't exist
  - False Negative tracks (FNs) – missing detections although the things are there
  - Identity Switches (IDsw) – confusing identity between objects
  - Fragmentation (Frag) – Ground truth coverage cut (i.e. no tracks assigned to ground truth for some frames)

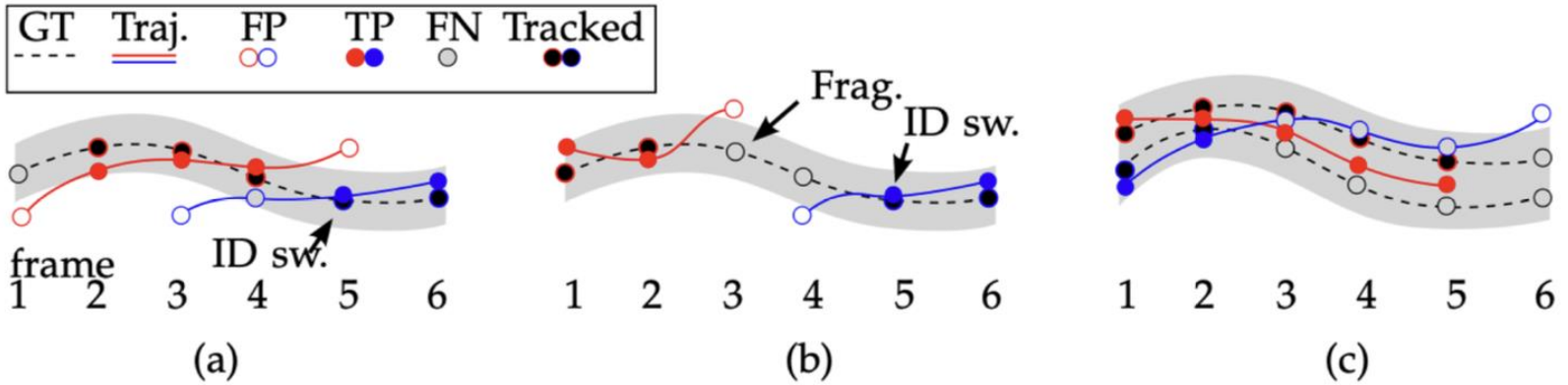
- **Metric: Multi-Object Tracking Accuracy MOTA**

$$\mathrm{MOTA} = 1 - \frac{\sum_t (\mathrm{FN}_t + \mathrm{FP}_t + \mathrm{IDsw}_t + \mathrm{Frag}_t)}{\sum_t \mathrm{GT}_t}.$$

Ground truth objects

Evaluation Metrics

- How do we compute identity switches?



a) An ID switch is counted at frame 5 because the ground truth track is assigned first to the red reconstructed track, then to the blue.

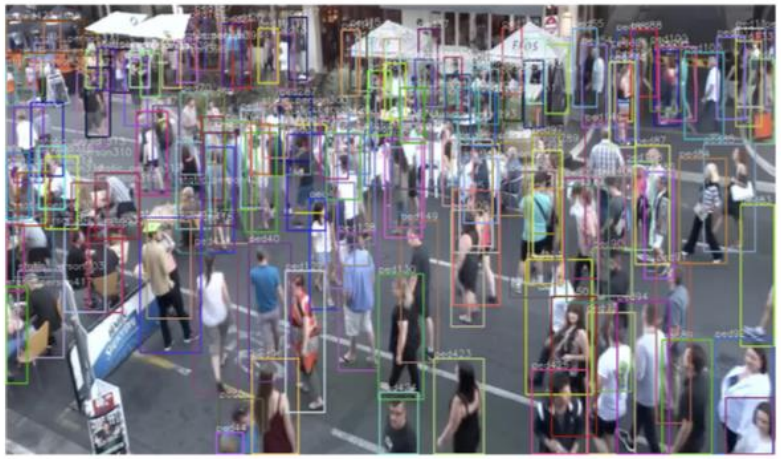
b) An ID switch at frame 5 and a fragmentation at frames 3 and 4.

c) Identity is preserved. If two trajectories overlap within a threshold with a ground truth trajectory, the one that forces least ID switches is chosen (in this case, the red one).

Datasets

- **MOTChallenge: www.motchallenge.net (people)**
  - Several challenges from less to more crowded

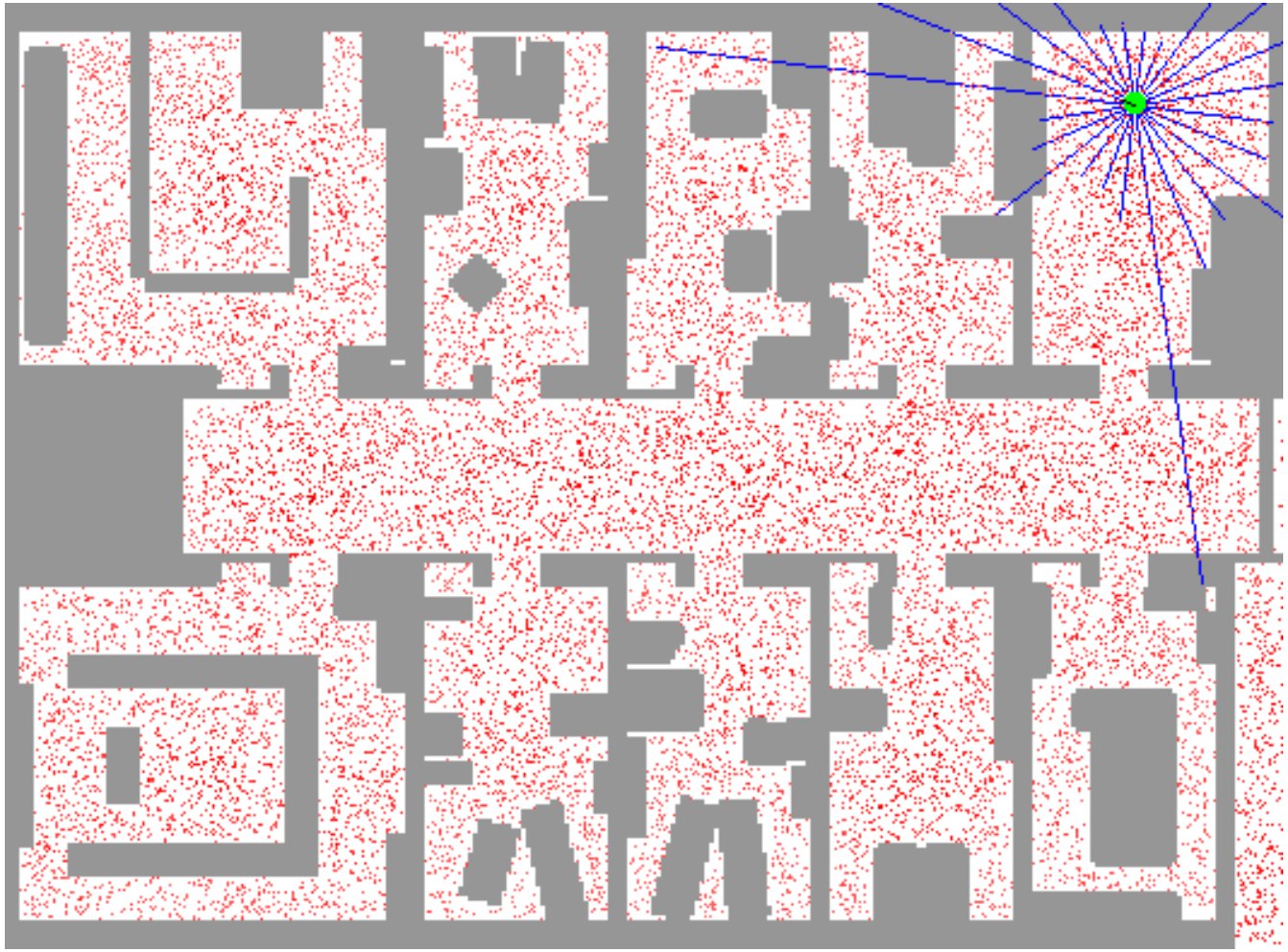




- **KITTI benchmark: http://www.cvlibs.net/datasets/kitti/ (vehicles)**
- **UA-Detrac: http://detrac-db.rit.albany.edu (vehicles)**

Particle Filters (a Glimpse)

Motivating example: Robot localization using maps and sonar



Literature / Links

1. **Battaglia et al.: *Relational inductive biases, deep learning, and graph networks*. 2018, https://arxiv.org/abs/1806.01261**
2. **G.Brasò, L. Leal-Taixé. *Learning a Neural Solver for Multiple Object Tracking*. 2019, https://arxiv.org/abs/1912.07515**



Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!